

Lezione 6 – Select

Reti di calcolatori

Modulo 4 - Tecniche di programmazione distribuita

Unità didattica 1 - Socket Library

Ernesto Damiani

Università degli Studi di Milano - Ssri - CDL ONLINE

Chiamate bloccanti

- **Molte delle funzioni esaminate bloccano il programma che le invoca fino a un determinato evento:**
 - **accept**: fino a che non arriva una richiesta di connessione;
 - **connect**: fino a che non viene stabilita la connessione;
 - **recv**, **recvfrom**: fino a che non viene ricevuto un pacchetto (di dati);
 - **send**, **sendto**: fino a che i dati non vengono immessi nel buffer del socket.
- **Per programmi semplici il blocco è utile.**
- **Nel caso di programmi più complessi diventa un problema:**
 - connessioni multiple;
 - invii e connessioni simultanee.

Chiamate non bloccanti

Opzioni:

- usare chiamate bloccanti, ma scrivere codice multi-thread o multiprocesso;
- disattivare la funzione di blocco, per esempio usando la funzione di controllo del descrittore di file `fcntl`;
- usare la chiamata `select`.

Chiamata `select` ⁽¹⁾

- **Può essere permanentemente bloccante, bloccante per un tempo limitato o non bloccante.**
- **Input: una serie di descrittori di file (socket).**
- **Output: informazioni sullo stato dei descrittori di file (socket).**
- **Identifica i socket che sono “pronti per l'uso”, su cui sono disponibili dei dati.**

Chiamata select (2)

- **Usa la struttura `struct fd_set`**
 - È un vettore di bit.
 - Se il bit *i* è impostato a `[readfds, writefds, exceptfds]`, `select` controllerà se il descrittore di file (cioè il socket) *i* è pronto per `[lettura, scrittura, eccezione]`.
- **Prima di chiamare `select`**
 - `FD_ZERO(&fdvar)`: inizializza la struttura;
 - `FD_SET(i,&fdvar)`: per controllare il descrittore di file *i*.
- **Dopo aver chiamato `select`**
 - `int FD_ISSET(i, &fdvar)`: booleano, restituisce `TRUE` se *i* è “pronto”.

Chiamata della funzione select

- **`int status = select(nfds, &readfds, &writefds, &exceptfds, &timeout);`**
 - `status`: numero di oggetti pronti, altrimenti -1;
 - `nfds`: 1 + numero massimo di descrittori da controllare;
 - `readfds`: elenco di descrittori da controllare perchè pronti alla lettura;
 - `writefds`: elenco di descrittori da controllare perchè pronti alla scrittura;
 - `exceptfds`: elenco di descrittori da controllare perchè si è verificata un'eccezione;
 - `timeout`: tempo dopo il quale `select` ritorna, anche se nessun socket è pronto, (può essere 0 o ∞ , in questo caso il parametro di timeout punta a `NULL`).

Altre funzioni utili

- **bzero(char* c, int n)**
copia *n* byte iniziali nulli (0) in *c*.
- **gethostname(char *name, int len)**
Restituisce il nome dell'host corrente.
- **gethostbyaddr(char *addr, int len, int type)**
Restituisce l'indirizzo, formattato rispetto al tipo *type* (per internet *AF_INET*), dell'host specificato in *addr*.
- **inet_addr(const char *cp)**
Converte stringhe di caratteri decimali puntati, rappresentanti un indirizzo IP, in interi lunghi.
- **inet_ntoa(const struct in_addr in)**
Converte **long** in notazione decimale puntata.

Porte libere

- **Talvolta, in caso di un'uscita "rude" da un programma (Ctrl-c), il sistema operativo non libera immediatamente la porta.**
- **Alla fine, solo dopo alcuni minuti, la porta verrà liberata.**
- **Per ridurre le probabilità che si verifichi questo problema, bisogna inserire il seguente codice:**

```
#include <signal.h>
void cleanExit(){exit(0);}
- nel codice che usa il socket:
    ▪ signal(SIGTERM, cleanExit);
    ▪ signal(SIGINT, cleanExit);
```

